Red Hat OpenShift AI Self-Managed 3.4

Creating distributed data processing applications with the Kubeflow Spark Operator

Creating distributed data processing applications with KSO Red Hat OpenShift AI Self-Managed

Last Updated: 2026-05-15

Red Hat OpenShift AI Self-Managed 3.4 Creating distributed data processing applications with the Kubeflow Spark Operator

Creating distributed data processing applications with KSO Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

As a cluster administrator, you can use the Kubeflow Spark Operator to create data processing applications in distributed environments

Table of Contents

CHAPTER 1. OVERVIEW OF THE KUBEFLOW SPARK OPERATOR (KSO)

The Kubeflow Spark Operator allows you to run Spark data processing applications in a distributed environment on Red Hat OpenShift AI. You can create custom resources (CRs) for specifying, running and surfacing Spark applications.

KSO for Apache Spark currently supports the following:

- Spark versions 4.0.1 and above.
- When a **SparkApplication** custom resource is created, the operator creates and runs a **spark-submit** job that starts Spark driver and executor pods.
- Native Cron support for running scheduled applications.
- Customization of Spark pods, including: mounting ConfigMaps/volumes and setting pod affinity.
- Automatic application restart with custom policies.
- Collecting and exporting application metrics and driver metrics to Prometheus.

1.1. KUBEFLOW SPARK OPERATOR (KSO) ARCHITECTURE

The Spark operator consists of the following:

- **SparkApplication Controller:** A job that watches Create, Update and Delete events in **SparkApplication** resources.
- **Submission Runner:** When a **SparkApplication** custom resource is created, the operator creates and runs a **spark-submit** job that starts Spark driver and executor pods.
- **Spark Pod Monitor:** Observes the **Driver** and **Executor** pods and updates the **.status** field of the **SparkApplication** CR.
- **Mutating Admission Webhook:** A component that intercepts pod creation requests and injects ConfigMap mounts or Volumes into the **Driver** and **Executor** pods before they are scheduled.

CHAPTER 2. ACTIVATING THE KUBEFLOW SPARK OPERATOR

You can activate the Kubeflow Spark Operator on your OpenShift cluster by setting its **managementState** to **Managed** in the OpenShift AI Operator **DataScienceCluster** custom resource (CR).

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have cluster administrator privileges.
- You have installed the OpenShift CLI (**oc**).
- You have installed the Red Hat OpenShift AI Operator on your cluster.
- You have an existing Spark image that you can import to you OpenShift AI cluster.

Procedure

1. Log in to the OpenShift web console as a cluster administrator.
2. In the **Administrator** perspective, click **Ecosystem → Installed Operators**.
3. Click the **Red Hat OpenShift AI Operator** to open its details.
4. Click the **DataScienceCluster** tab.
5. Click the **YAML** tab.
   An embedded YAML editor opens, displaying the configuration for the **DataScienceCluster** custom resource.
6. In the YAML editor, locate the **spec.components** section, navigate the **kubeflowsparkoperator** parameter and update the field.

```
spec:
  components:
    kubeflowsparkoperator:
      managementState: Managed
```

7. Click **Save** to apply your changes.

Verification

After you activate the Kubeflow Spark Operator, you can verify that it is running in your cluster:

1. In the OpenShift web console, click **Workloads → Pods**.
2. From the **Project** list, select the **redhat-ods-applications** namespace.
3. Confirm that a pod with the label **app.kubernetes.io/name=kubeflow-spark-operator** is displayed and has a status of **Running**.

CHAPTER 3. USING THE KUBEFLOW SPARK OPERATOR

You can run Spark data processing applications in a distributed environment on Red Hat OpenShift AI by using the Kubeflow Spark Operator. You can create custom resources (CRs) for specifying, running and surfacing Spark applications.

The following procedure explain how to create a custom Spark image and a **SparkApplication** custom resource (CR) application.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have cluster administrator privileges.
- You have installed the Kubeflow Spark Operator on your OpenShift AI instance.

Procedure

1. Create a **ContainerFile** called **spark-image** with the following parameters:

```
FROM apache/spark:4.0.1

LABEL description="Apache Spark image compatible with OpenShift restricted-v2 SCC"

USER root

RUN chgrp -R 0 /opt/spark && \
    chmod -R g=u /opt/spark && \
    mkdir -p /opt/spark/work-dir /opt/spark/logs && \
    chgrp -R 0 /opt/spark/work-dir /opt/spark/logs && \
    chmod -R 775 /opt/spark/work-dir /opt/spark/logs

RUN chmod 1777 /tmp

ENV HOME=/home/spark

RUN mkdir -p /home/spark && \
    chgrp -R 0 /home/spark && \
    chmod -R g=u /home/spark && \
    chmod 775 /home/spark
```

2. Build and tag the image for your registry with the following command:

```
$ podman build -t quay.io//spark-image:4.0.1 -f spark-image
```

3. Push your **spark-image** to the registry:

```
$ podman push quay.io//spark-image:4.0.1
```

4. The **SparkApplication** custom resource definition (CRD) allows you to define Spark applications using YAML manifests.

```
$ oc apply -f sparkapplication-example.yaml
```

Example **SparkApplication** custom resource (CR)

```
apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: ${APP_NAME}
  namespace: ${APP_NAMESPACE}
spec:
  type: Scala
  mode: cluster
  image: <your-custom-spark-image>
  imagePullPolicy: IfNotPresent
  mainClass: org.apache.spark.examples.SparkPi
  mainApplicationFile: local:///opt/spark/examples/jars/spark-examples.jar
  arguments:
    - "1000"
  sparkVersion: "4.0.1"
  restartPolicy:
    type: Never
  driver:
    cores: 1
    coreLimit: "1200m"
    memory: "512m"
    labels:
      version: 4.0.1
    serviceAccount: spark-operator-spark
    securityContext: {}
  executor:
    cores: 1
    instances: 1
    memory: "512m"
    labels:
      version: 4.0.1
    securityContext: {}
```

Table 3.1. **SparkApplication** CR reference

Field	Description
namespace	The namespace to run the Spark application, the default is the **redhat-ods-applications** namespace. If you want to run a Spark application on a custom namespace, you need to adjust the role binding to match the roles of the default namespace. For more information
type	The language of the application. For example, Python, Scala, etc.
mode	Deployment mode, example fields include, **cluster** or **client**. In cluster mode, the driver runs in a pod.
image	The container image to use for the driver and executors. This image must include the Spark v4.0.1 runtime.

Field	Description
mainApplicationFile	The entry point path. For example, **local:///app/scripts/run_spark_job.py**.
sparkVersion	The version of Spark to use, the version included must match the image.
restartPolicy	Handling of failures. Example fields include: **Never**, **OnFailure**, **Always**.
driver/executor	Resource requests (cores, memory), labels, service accounts, and security contexts.
volumes/volumeMounts	PVCs for input and output data.
serviceAccount	The spark-operator-spark service account is created automatically by the Spark Operator installation and includes the necessary RBAC permissions for Spark drivers to create and manage executor pods.

NOTE

When using Spark example YAMLs in OpenShift AI, the **runAsGroup** and **runAsUser** parameters must be removed, they may cause jobs crashed in your environment.

3.1. USING THE KUBEFLOW SPARK OPERATOR IN A CUSTOM NAMESPACE

You can utilize the Kubeflow Spark Operator in a custom namespace. The following documentation explains how to view the **Role**, **RoleBinding**, and **ServiceAccount** resources in the default **redhat-ods-applications** and add them to a custom namespace.

Prerequisite

- You have installed OpenShift 4.19 or newer.
- You have cluster administrator privileges.
- You have installed the Kubeflow Spark Operator on your OpenShift AI instance.
- You have installed the OpenShift CLI (**oc**).

Procedure

1. To get the **ServiceAccount** resource YAML, run the following command:

```
$ oc get serviceaccount spark-operator-spark -n redhat-ods-applications -o yaml
```

   Then, update the **namespace:** field to your custom namespace.

2. To get the **Role** resource YAML, run the following command:

```
$ oc get role spark-operator-role -n redhat-ods-applications -o yaml
```

   Then, update the **namespace:** field to your custom namespace.

3. To get the **RoleBinding** resource YAML, run the following command:

```
$ oc get rolebinding spark-operator-rolebinding -n redhat-ods-applications -o yaml
```

   Then, update the two **namespace:** fields to your custom namespace.

4. Save the new **Role**, **RoleBinding**, and **ServiceAccount** resources to a file or multiple files.

5. Apply the changes to your custom namespace with the following command:

```
$ oc apply -f <resources-file(s).yaml> -n <custom-namespace>
```